📘 Programs 1–10: Concepts + Functions Used

🔹 PROGRAM 1 — Creating Arrays and Array Operations

The program creates 1D/2D arrays, calculates statistics, and normalizes an array.

Concept Used

- NumPy arrays
- 1D and 2D arrays
- Random number generation
- Mean and Standard Deviation
- Array normalization
- Array arithmetic

Libraries

import numpy as np

Functions Used

Function	Purpose
np.arange(10)	Creates numbers from 0 to 9
np.random.rand(3,4)	Creates a 3×4 array of random values
np.array()	Creates a NumPy array
input()	Takes input from user
int()	Converts input to integer
float()	Converts input to floating-point number
np.mean()	Calculates mean
np.std()	Calculates standard deviation

Important Operation

normalized = (arr - mean) / std

Concept: Standardization/normalization of data.

🧠 Viva one-liner

This program demonstrates basic NumPy array creation, statistical operations, and normalization.

🔹 PROGRAM 2 — Generating Array, Reshaping and Stacking

This program creates evenly spaced values, an identity matrix, reshapes arrays, and stacks matrices.

Concept Used

- NumPy arrays
- Evenly spaced values
- Identity matrix
- Reshaping
- Vertical stacking
- Horizontal stacking

Library

import numpy as np

Functions Used

Function	Purpose
np.linspace()	Creates evenly spaced values
np.eye()	Creates an identity matrix
np.array()	Creates NumPy array
input()	Gets user input
int()	Converts input to integer
.reshape()	Changes array dimensions
np.vstack()	Stacks arrays vertically
np.hstack()	Stacks arrays horizontally

Important Examples

np.linspace(0, 1, 10)

→ 10 evenly spaced values between 0 and 1.

array.reshape(2,5)

→ Converts a 1D array into a 2×5 array.

np.vstack((a,b))

→ Places one matrix below another.

np.hstack((a,b))

→ Places matrices side by side.

🧠 **Viva one-liner**

This program demonstrates array generation, reshaping and combining arrays using NumPy.

🔹 **PROGRAM 3 — Indexing, Slicing & Matrix Operations**

This program extracts array elements, reverses an array, performs matrix multiplication, and calculates a determinant.

Concept Used

- Array indexing
- Array slicing
- Array reversal
- Matrix multiplication
- Determinant

Library

import numpy as np

Functions / Operations Used

Function / Operation	Purpose
np.array()	Creates array

Function / Operation	Purpose
input()	Gets user input
map()	Applies conversion to each input value
int()	Converts to integer
.split()	Splits input into elements
.reshape()	Changes dimensions
[::-1]	Reverses array
@	Matrix multiplication
np.linalg.det()	Calculates determinant

Important Concepts

Third column:

a[:,2]

: → all rows
2 → third column because indexing starts from 0.

Reverse:

arr[::-1]

Matrix multiplication:

A @ B

Determinant:

np.linalg.det(M)

🧠 **Viva one-liner**

This program demonstrates NumPy indexing, slicing, matrix multiplication and determinant calculation.

🔹 **PROGRAM 4 — Statistical Operations & Broadcasting**

This program calculates row/column means, finds minimum/maximum values, adds a constant to an array, and performs row-wise multiplication.

Concept Used

- Statistical operations
- Axis
- Broadcasting
- Array arithmetic
- Row-wise operations

Library

import numpy as np

Functions Used

Function	Purpose
np.random.randint()	Generates random integers
np.mean()	Calculates mean
np.min()	Finds minimum
np.max()	Finds maximum
np.array()	Creates array
input()	Gets input
map()	Converts multiple input values
.reshape()	Changes array shape

Important Concept — axis

np.mean(arr, axis=1)

→ Mean of each **row**

np.mean(arr, axis=0)

→ Mean of each **column**

Broadcasting

a + c

The constant c is automatically added to every element.

m * k.reshape(r,1)

Different constants are multiplied with different rows.

🧠 **Viva one-liner**

This program demonstrates statistical operations and NumPy broadcasting without explicitly using loops.

🔹 **PROGRAM 5 — Creating a Pandas DataFrame**

This program creates a DataFrame from a list of student data.

Concept Used

- Pandas
- DataFrame
- Tabular data
- Rows and columns

Library

import pandas as pd

Functions Used

Function	Purpose
pd.DataFrame()	Creates a DataFrame
print()	Displays the DataFrame

Important Code

df = pd.DataFrame(data, columns=["Name", "Age", "Marks"])

This creates a table with:

- Name
- Age
- Marks

🧠 **Viva one-liner**

This program demonstrates how to create and display a Pandas DataFrame from a list of data.

🔹 PROGRAM 6 — Dataset Analysis using Pandas and Matplotlib

The program creates a student dataset, performs basic statistical analysis, and displays a bar graph.

Concepts Used

- Pandas DataFrame
- Data analysis
- Mean
- Maximum
- Minimum
- Data visualization
- Bar chart

Libraries

import pandas as pd

import matplotlib.pyplot as plt

Functions Used

Function	Purpose
pd.DataFrame()	Creates DataFrame
.mean()	Calculates average
.max()	Finds highest value
.min()	Finds lowest value
plt.bar()	Creates bar graph
plt.title()	Adds graph title
plt.xlabel()	Labels X-axis

Function	Purpose
plt.ylabel()	Labels Y-axis
plt.show()	Displays graph

Important Code

df["Marks"].mean()

→ Average marks

df["Marks"].max()

→ Highest marks

df["Marks"].min()

→ Lowest marks

plt.bar(df["Name"], df["Marks"])

→ Creates bar chart.

🧠 **Viva one-liner**

This program uses Pandas for dataset analysis and Matplotlib for visualizing student marks.

🔹 **PROGRAM 7 — Mean, Median, Mode, Variance & Standard Deviation**

This program calculates five important statistical measures from a dataset.

Concepts Used

- Descriptive statistics
- Mean
- Median
- Mode
- Variance
- Standard deviation
- Frequency/count

Library

import numpy as np

Functions Used

Function	Purpose
np.array()	Creates dataset
np.mean()	Calculates mean
np.median()	Calculates median
np.unique()	Finds unique values
return_counts=True	Returns frequency of each value
np.argmax()	Finds index of maximum value
np.var()	Calculates variance
np.std()	Calculates standard deviation

Mode Logic

values, counts = np.unique(data, return_counts=True)

mode = values[np.argmax(counts)]

Meaning:

1. Find unique values.
2. Count how many times each occurs.
3. Find the largest count.
4. Select the corresponding value.

🧠 **Viva one-liner**

This program calculates the major measures of central tendency and dispersion using NumPy.

🔹 **PROGRAM 8 — Histogram**

This program plots a histogram for the given dataset.

Concepts Used

- Data visualization
- Histogram
- Frequency distribution
- Class intervals
- Bins

Libraries

import numpy as np

import matplotlib.pyplot as plt

Functions Used

Function	Purpose
np.array()	Creates dataset
plt.hist()	Creates histogram
plt.title()	Adds title
plt.xlabel()	Labels X-axis
plt.ylabel()	Labels Y-axis
plt.show()	Displays graph

Important

plt.hist(data, bins=[10,15,20,25,30,35])

bins → defines the class intervals.

What does histogram show?

It shows the **frequency/distribution of numerical data** across intervals.

🧠 **Viva one-liner**

A histogram represents the frequency distribution of continuous or numerical data using bins.

🔹 **PROGRAM 9 — Box-and-Whisker Plot**

This program creates a box-and-whisker plot for the given dataset.

Concept Used

- Data visualization
- Box plot
- Data distribution
- Quartiles
- Median
- Spread of data

Library

import matplotlib.pyplot as plt

Functions Used

Function	Purpose
plt.boxplot()	Creates box-and-whisker plot
plt.title()	Adds title
plt.ylabel()	Labels Y-axis
plt.show()	Displays plot

Important Concept

A box plot represents:

- Minimum
- Q1 — First Quartile
- Median
- Q3 — Third Quartile
- Maximum

It can also help identify **outliers**.

🧠 **Viva one-liner**

A box-and-whisker plot is used to visualize the distribution, spread and quartiles of a dataset.

🔹 PROGRAM 10 — Line Plot and Bar Chart

This program creates both a line plot and a bar chart for elapsed time and speed.

Concepts Used

- Data visualization
- Line graph
- Bar chart
- X-axis and Y-axis
- Relationship between variables

Library

import matplotlib.pyplot as plt

Functions Used

Function	Purpose
plt.figure()	Creates a new figure
plt.plot()	Creates line plot
plt.bar()	Creates bar chart
plt.title()	Adds title
plt.xlabel()	Labels X-axis
plt.ylabel()	Labels Y-axis
plt.grid()	Adds grid
plt.show()	Displays graph

Line Plot

plt.plot(elapsed_time, speed, marker='o')

Shows how **speed changes with elapsed time**.

Bar Chart

plt.bar(elapsed_time, speed)

Represents each time-speed value using bars.

🧠 Viva one-liner

This program demonstrates two common visualization techniques—line plots and bar charts—using Matplotlib.

⚡ MOST IMPORTANT FUNCTIONS TO MEMORIZE

If your practical exam asks **"What functions did you use?"**, these are the ones you should know:

NumPy

Function	Remember as
np.array()	Create array
np.arange()	Sequence
np.linspace()	Evenly spaced values
np.random.rand()	Random decimal values
np.random.randint()	Random integers
np.mean()	Mean
np.median()	Median
np.std()	Standard deviation
np.var()	Variance
np.min()	Minimum
np.max()	Maximum
np.unique()	Unique values
np.argmax()	Index of maximum

Function	Remember as
np.eye()	Identity matrix
np.vstack()	Vertical stack
np.hstack()	Horizontal stack
np.linalg.det()	Determinant

Pandas

Function	Remember as
pd.DataFrame()	Create table/DataFrame
pd.read_csv()	Read CSV
.mean()	Average
.max()	Highest
.min()	Lowest

Matplotlib

Function	Remember as
plt.plot()	Line graph
plt.bar()	Bar graph
plt.hist()	Histogram
plt.boxplot()	Box plot
plt.scatter()	Scatter plot
plt.title()	Graph title
plt.xlabel()	X-axis label
plt.ylabel()	Y-axis label
plt.grid()	Grid
plt.show()	Display graph

📘 PROGRAM 11 — FIND-S Algorithm

Concept Used

- Machine Learning
- Concept learning
- Hypothesis
- FIND-S algorithm
- Most specific hypothesis

⚠️ Important

Your document's **aim says FIND-S**, but the code shown underneath is actually a **Linear Regression** program using LinearRegression().

So for your **actual code**, the concepts/functions are:

Libraries

import numpy as np

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression

Functions Used

Function	Purpose
np.array()	Creates NumPy array
.reshape()	Changes array shape
LinearRegression()	Creates linear regression model
.fit()	Trains the model
.predict()	Predicts output
plt.scatter()	Creates scatter plot
plt.plot()	Draws regression line
plt.xlabel()	X-axis label
plt.ylabel()	Y-axis label

Function	Purpose
plt.title()	Graph title
plt.show()	Displays graph

🧠 **Viva**

The shown code actually implements simple linear regression, not FIND-S.

🔹 **PROGRAM 12 — Simple Linear Regression**

The program implements simple linear regression and plots the graph.

Concept Used

- Supervised learning
- Regression
- Simple linear regression
- Training a model
- Prediction
- Regression line

Library

from sklearn.linear_model import LinearRegression

Also:

import numpy as np

import matplotlib.pyplot as plt

Functions Used

Function	Purpose
np.array()	Creates data
.reshape()	Converts data into required shape
LinearRegression()	Creates regression model

Function	Purpose
.fit()	Trains model
.predict()	Predicts values
plt.scatter()	Displays original data points
plt.plot()	Displays regression line
plt.show()	Shows graph

Important Concept

Simple linear regression finds a relationship between:

Independent variable X → Dependent variable Y

Basic equation:

Y = mX + c

🧠 **Viva**

Simple linear regression predicts a continuous output based on one independent variable.

🔹 **PROGRAM 13 — Single Layer Perceptron**

Your document gives the aim as implementing a **single-layer perceptron**.

Concept Used

- Artificial Neural Network
- Perceptron
- Supervised learning
- Binary classification
- Weights
- Bias
- Activation function

Main Components

Input → Weighted Sum → Activation Function → Output

Important Terms

Term	Meaning
Input	Data supplied to neuron
Weight	Importance given to input
Bias	Additional value added to weighted sum
Activation function	Decides output
Output	Prediction of perceptron

🧠 **Viva**

A single-layer perceptron is a basic neural network used mainly for binary classification.

🔹 **PROGRAM 14 — Naïve Bayes Classifier**

The program reads a CSV dataset, converts categorical data into numbers, trains a Naïve Bayes classifier, predicts test data, and calculates accuracy.

Concept Used

- Supervised Machine Learning
- Classification
- Naïve Bayes
- Probability
- Categorical data
- Train-test split
- Accuracy

Libraries

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder

from sklearn.naive_bayes import CategoricalNB

from sklearn.metrics import accuracy_score

Functions Used

Function	Purpose
pd.read_csv()	Reads CSV file
.iloc[]	Selects rows/columns
LabelEncoder()	Converts text categories to numbers
.fit_transform()	Learns categories and converts them
train_test_split()	Splits dataset into training/testing data
CategoricalNB()	Creates Naïve Bayes classifier
.fit()	Trains model
.predict()	Predicts test values
accuracy_score()	Calculates accuracy
print()	Displays results

Important Concept

Naïve Bayes is based on **Bayes' theorem** and assumes features are conditionally independent.

Flow

CSV → Encode data → Split → Train → Predict → Calculate Accuracy

🧠 **Viva**

Naïve Bayes is a probabilistic classification algorithm based on Bayes' theorem.

🔹 **PROGRAM 15 — Decision Tree using ID3**

This program demonstrates a decision tree using the **ID3 algorithm** and entropy.

Concept Used

- Supervised learning

- Classification
- Decision Tree
- ID3 algorithm
- Entropy
- Information gain
- Label encoding

Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder
```

Functions Used

Function	Purpose
pd.DataFrame()	Creates DataFrame
LabelEncoder()	Converts categorical values to numbers
.fit_transform()	Encodes data
DecisionTreeClassifier()	Creates decision tree
.fit()	Trains tree
.predict()	Predicts new sample
plot_tree()	Displays decision tree
plt.figure()	Creates figure
plt.title()	Adds title
plt.show()	Displays tree

Important Code

```
model = DecisionTreeClassifier(criterion="entropy")
```

criterion="entropy" → uses **entropy** for splitting.

🧠 Viva

ID3 builds a decision tree by selecting attributes based on entropy and information gain.

🔹 PROGRAM 16 — K-Nearest Neighbor (KNN)

The aim is to classify the **Iris dataset** using K-Nearest Neighbor and print correct and wrong predictions.

Concept Used

- Supervised learning
- Classification
- KNN
- Distance measurement
- Nearest neighbors
- Iris dataset

Main Idea

KNN classifies a new data point based on the classes of its **K nearest data points**.

Important Terms

Term	Meaning
K	Number of neighbors considered
Neighbor	Closest data point
Distance	Measures closeness
Classification	Assigning a class

Common Functions/Methods

For a typical implementation:

Function	Purpose
load_iris()	Loads Iris dataset

Function	Purpose
train_test_split()	Splits data
KNeighborsClassifier()	Creates KNN model
.fit()	Trains model
.predict()	Predicts classes
accuracy_score()	Calculates accuracy

🧠 **Viva**

KNN classifies a data point based on the majority class among its K nearest neighbors.

🔹 **PROGRAM 17 — K-Means Clustering**

The program reads a CSV file and groups the data into **two clusters** using K-Means.

Concept Used

- Unsupervised learning
- Clustering
- K-Means algorithm
- Centroids
- Distance
- Data visualization

Libraries

import pandas as pd

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt

Functions Used

Function	Purpose
pd.read_csv()	Reads CSV file
KMeans()	Creates K-Means model
.fit_predict()	Fits model and returns cluster labels
plt.scatter()	Plots clusters
plt.xlabel()	X-axis label
plt.ylabel()	Y-axis label
plt.title()	Graph title
plt.show()	Displays graph

Important Code

model = KMeans(n_clusters=2, random_state=42)

n_clusters=2 → creates **2 clusters**.

data["Cluster"] = model.fit_predict(X)

→ Assigns each data point to a cluster.

🧠 **Viva**

K-Means is an unsupervised learning algorithm that groups similar data points into K clusters.

🔹 **PROGRAM 18 — Artificial Neural Network using Backpropagation**

The aim is to build an Artificial Neural Network using the **Backpropagation algorithm** and test it with suitable data.

Concept Used

- Artificial Neural Network
- Backpropagation
- Forward propagation
- Error calculation

- Weight adjustment
- Training
- Neural network learning

Important Terms

Term	Meaning
Input layer	Receives input data
Hidden layer	Processes information
Output layer	Produces prediction
Weight	Controls strength of connection
Bias	Additional adjustable value
Activation function	Determines neuron output
Error	Difference between actual and predicted output
Backpropagation	Updates weights to reduce error

Basic Flow

Input → Forward Propagation → Output → Calculate Error → Backpropagation → Update Weights → Repeat

🧠 **Viva**

Backpropagation trains a neural network by calculating the error and adjusting the weights to reduce that error.

🔥 **Programs 11–18 — Super Short Revision**

Program	Main Concept	Most Important Function/Class
11	FIND-S *(aim)* / Linear Regression *(shown code)*	LinearRegression()
12	Simple Linear Regression	.fit(), .predict()
13	Single Layer Perceptron	Perceptron

Program	Main Concept	Most Important Function/Class
14	Naïve Bayes	CategoricalNB()
15	ID3 Decision Tree	DecisionTreeClassifier(criterion="entropy")
16	KNN	KNeighborsClassifier()
17	K-Means	KMeans()
18	Backpropagation ANN	Backpropagation

⭐ **Algorithms you should definitely remember**

FIND-S → Most specific hypothesis

Linear Regression → Predict continuous values

Perceptron → Binary classification

Naïve Bayes → Probability-based classification

ID3 → Decision tree using entropy

KNN → Classification using nearest neighbors

K-Means → Unsupervised clustering

Backpropagation → Train neural networks by reducing error